DECENTRALISED MULTI-AGENT SEARCH AND RESCUE UNDER LIMITED COMMUNICATION

Boinapally Dhanvini
ID: 24110079
Branch: Artificial Intelligence
boinapally.dhanvini@iitgn.ac.in

Bollineni Snehitha Rao
ID: 24110081
Branch: Artificial Intelligence
snehitha.bollineni@iitgn.ac.in

Jils Shah
ID: 24110148
Branch: Computer Science
jils.shah@iitgn.ac.in

Thaker Shivangi Pareshkumar
ID: 24110371
Branch: Computer Science
shivangi.thaker@iitgn.ac.in

ABSTRACT

A person is trapped at an unknown location inside a hazardous building, and a team of heterogeneous autonomous agents must locate them and bring them to an exit. Search drones sense well but cannot perform the rescue; ground rescue robots can perform the rescue but sense poorly, so information must travel from the agents that locate the victim to the agents that can reach them. Communication, however, is a limited resource. We model the building as a graph with obstacles, hazards, an exit and one hidden victim; the floor plan itself is unknown and discovered by sensing, agents keep Bayesian beliefs built from noisy local scans, and they allocate tasks under a team communication budget B. A centralised baseline solves the task-allocation CSP exactly with the Hungarian method on pooled information; the decentralised method solves the corresponding DCOP with MGM or DSA-C over a range-limited communication graph. Over 30,600 simulated episodes we find that (i) two messages per agent—the drone-to-robot handoff—recover 90% of the success lost without communication (success $0.66 \rightarrow 0.96$); (ii) with unrestricted budget the decentralised team succeeds in 99% of runs using 48% of the centralised message volume, but is 21% slower, half of that gap being due to having to discover and share the map; (iii) communication is worth more when sensors are noisy, while intermediate budgets let partial, noisy evidence spread and make robots chase false alarms; and (iv) coordination messages pay off only with a large budget, where the cheaper DSA-C matches or beats MGM.

Keywords Multi-agent systems Search and rescue Decentralised coordination Task allocation Constraint satisfaction Distributed constraint optimisation Limited communication

1 Introduction

Search-and-rescue operations in hazardous buildings are difficult because responders often do not know where a trapped person is, or which routes remain safe. Autonomous robots can support such operations by exploring, sensing and mapping, and multi-robot teams have been studied extensively for exactly this setting [1]. Teams that mix robots with different capabilities have also been studied for urban search and rescue, where the question of which robot should take which task becomes central [3].

This project considers a team of heterogeneous agents inside a simulated hazardous building: *search drones*, which move quickly and sense over a wide area but cannot rescue anyone, and *ground rescue robots*, which move only along accessible ground routes and sense poorly but can carry the victim to an exit. The split is deliberate: the agents that find the victim are never the agents that can

rescue the victim, so a report has to travel from one group to the other. Three properties make this a multi-agent problem rather than a path-planning one: no agent observes the whole environment; capabilities are complementary, so tasks must be allocated to agents able to perform them [4]; and communication is limited, so agents must decide not only where to go but also what is worth saying. Such coordination is naturally modelled as a distributed constraint optimisation problem (DCOP) [2].

Research question. *How does reducing the communication budget affect decentralised coordination in multi-agent search and rescue when agents have incomplete and noisy information?* Secondary questions are whether communication becomes more or less valuable as sensors become less reliable, how far decentralised coordination stays from a centralised coordinator with pooled information, and how much redundant exploration [6] appears when agents cannot talk. This report completes the project review with the methodology (Section 4), the results (Section 5) and the individual contributions (Section 6).

2 Preliminaries

Agents and heterogeneity. $\mathscr{A} = \mathscr{D} \cup \mathscr{R}$, $\mathscr{D} \cap \mathscr{R} = \emptyset$, where $\mathscr{D}$ are search drones and $\mathscr{R}$ ground robots. Drones have a larger sensing radius and higher reliability; only robots can rescue. **Environment.** The building is a graph $G = (V, E)$ generated as a 2D grid whose cells are free, obstacles or hazardous, with one exit v_{exit} and one victim location $v^\star$ unknown to the agents; time is discrete. **Incomplete information.** A *local observation* is what an agent's sensors return at its location; it is *noisy*: a scan may miss the victim or report one where there is none. **Tasks.** $\mathscr{T}$ is the set of candidate tasks (searching a region, carrying out the rescue) with rescue tasks $\mathscr{T}_{\text{res}} \subset \mathscr{T}$; *task allocation* assigns tasks to capable agents [4]. **Communication.** Agents exchange short messages; the total they may send in a run is limited by a budget B. **CSP and DCOP.** A CSP is a triple of variables, domains and constraints; a DCOP distributes the variables among agents and replaces constraints by cost functions that agents minimise by exchanging messages with neighbours [2].

3 Problem Formulation

Let $p_i^t \in V$ be the location of agent i at time t. **Decision variables:** $x_{ij} = 1$ if task $j \in \mathscr{T}$ is assigned to agent $i \in \mathscr{A}$ and 0 otherwise. **Assignment constraints:**

$$\sum_{i \in \mathscr{A}} x_{ij} \le 1 \;\; \forall j \in \mathscr{T}, \tag{1}$$

$$x_{ij} \le F_{ij} \;\; \forall i, j, \qquad F_{ij} = 0 \;\; \forall i \in \mathscr{D},\, j \in \mathscr{T}_{\text{res}}, \tag{2}$$

where $F_{ij} \in \{0, 1\}$ is agent i's capability for task j. **Observation model.** A scan by agent i returns, for every v within radius ρ_i, a reading $z_v \in \{0, 1\}$ with

$$\Pr(z_v = 1 \mid v = v^\star) = p_i^{\text{det}}, \qquad \Pr(z_v = 1 \mid v \neq v^\star) = p_i^{\text{fa}}, \tag{3}$$

with $p_i^{\text{det}} > p_{i'}^{\text{det}}$ and $\rho_i > \rho_{i'}$ for $i \in \mathscr{D}$, $i' \in \mathscr{R}$. **Communication constraint.** $C_{\text{comm}} = \sum_{t=1}^{T} \sum_{i \in \mathscr{A}} c(m_i^t) \le B$. **Objective.** With completion time T_{rescue}, movement cost C_{move} and success $S \in \{0, 1\}$ (T_{rescue} is set to the step limit when $S = 0$),

$$\min_{\{x_{ij}\}} \; J = \lambda_t T_{\text{rescue}} + \lambda_c C_{\text{comm}} + \lambda_m C_{\text{move}} + \lambda_f (1 - S) \quad \text{s.t. (1)–(2) and } C_{\text{comm}} \le B. \tag{4}$$

Decentralised form. The DCOP $\langle \mathscr{A}, X, D, F, \alpha \rangle$ has variables $X = \{x_{ij}\}$, domains $\{0, 1\}$, cost functions F derived from (4) and $\alpha(x_{ij}) = i$; each agent chooses its own variables from local observations and received messages. The centralised problem is the same optimisation with all variables controlled by one coordinator and C_{comm} not charged.

4 Methodology

Everything below is implemented in Python (NumPy/SciPy) in the accompanying repository; module names are given in `typewriter`. Both approaches run on the *same* simulator and the *same* randomly generated building for a given seed, and share the cost functions, so the only differences are who holds which information and how the allocation is solved.

4.1 Simulator (`sar/env.py`, `sar/agent.py`, `sar/mapknow.py`)

Building. An $n \times n$ grid ($n = 20$ by default) receives a few internal walls with door gaps (room structure), random debris obstacles (density 0.08) and hazard cells (density 0.06). The exit lies on the left wall; the victim is placed uniformly among robot-reachable cells at least $0.6n$ steps from the exit. Drones fly over hazards but not through walls; robots may not enter hazards. All agents enter at the exit (drones 2 cells/step, robots 1 cell/step). **Unknown floor plan.** As in the review, the state of unvisited locations is unknown: each agent holds its own terrain map in which every cell is *unknown* until it falls inside the agent's sensing window (or a teammate reports it). Agents plan shortest paths on this map *optimistically*—unknown cells are assumed traversable for their movement model—and re-plan every step; an agent that runs into an undiscovered wall or hazard learns it and re-routes. Cells

revealed to be obstacles or hazards are removed from the victim posterior. A *known-map* variant (the floor plan given from blueprints) is kept for comparison (E6). **Sensing and belief.** Drones scan a 5×5 window ($\rho = 2$), robots a 3×3 window ($\rho = 1$), with the reliability presets of Table 1. Each agent keeps a posterior $b_i(v) = \Pr(v^\star = v \mid \text{evidence})$, initially uniform over all cells (over robot-reachable cells when the map is known). Since there is a single victim, a reading z_v from a sensor with parameters $(p^{\text{det}}, p^{\text{fa}})$ multiplies $b(v)$ by the likelihood ratio

$$L(z_v) = \left(\frac{p^{\text{det}}}{p^{\text{fa}}}\right)^{z_v} \left(\frac{1-p^{\text{det}}}{1-p^{\text{fa}}}\right)^{1-z_v} \tag{5}$$

followed by renormalisation (all other cells share the same denominator). Updates are kept in log space. A robot that stands on a cell physically checks it: if the victim is there it picks them up, otherwise the cell's posterior is set to zero (certain absence). An agent *claims* the victim at $\hat{v} = \arg\max_v b_i(v)$ when $b_i(\hat{v}) \geq \tau = 0.7$.

4.2 Communication model and messaging rule (`sar/sim.py`)

A message is a broadcast received by every agent within Euclidean range $r_c = 9$ cells; each costs one unit ($c(m) = 1$). The team budget is split equally, each agent holding a quota $\lfloor B/|\mathscr{A}| \rfloor$, so $C_{\text{comm}} \leq B$ holds by construction and needs no global counter. Neighbour discovery (who is in range) is free. As promised in the review, *what* to send is fixed to one simple rule so that the budget remains the variable under study. Each agent buffers its unshared scans; an `OBS` message carries the whole buffer (so receivers apply exactly the update (5) with the sender's sensor parameters), the terrain it has discovered, the sender's current task, its cleared cells and its current claim. Messages are sent in priority order: (1) *urgent*: a new or retracted claim, a robot's report that a claimed cell was empty, or the "victim picked up" notice; (2) *delivery*: a drone holding its own claim that has not yet reached a robot flies towards the last known robot position and broadcasts once a robot is in range; (3) *relay*: a newly heard claim is re-broadcast once; (4) *periodic*: every 5 steps the observation buffer is shared; (5) DCOP messages (Section 4.5). Classes (4)–(5) are only sent while an agent has more than a reserve of 2 messages left, so the reserve is kept for victim reports. **Conflicting reports.** Each agent keeps the latest claim per sender and its own claim; it acts on the one with the highest confidence (ties: most recent), and discards claims on cells it knows to be empty.

4.3 Task set and cost functions (`sar/allocation.py`)

The map is split into $K = 16$ regions (4×4). The task set is $\mathscr{T} = \{\text{SEARCH}_k\}_{k=1}^{K} \cup \{\text{RESCUE}\} \cup \{\text{IDLE}\}$, where IDLE plays the role of "unassigned" and is exempt from (1). For agent i with belief b_i, speed s_i and distance $d_i(\cdot)$ in its own movement graph, the unary costs instantiate the time and failure terms of (4):

$$u_i(\text{SEARCH}_k) = \frac{d_i(k)}{s_i} - w_{\text{info}}\, q_i \sum_{v \in \text{region } k} b_i(v), \quad u_i(\text{RESCUE}) = \frac{d_i(\hat{v})}{s_i} - w_{\text{res}}\, b_i(\hat{v}), \quad u_i(\text{IDLE}) = 0, \tag{6}$$

where $q_i = p_i^{\text{det}}(2\rho_i{+}1)^2/25$ is the scanning efficiency and RESCUE exists only when the agent holds a claim $\hat{v}$. $u_i(\text{RESCUE}) = +\infty$ for drones encodes (2). The pairwise cost $w(x_i, x_j) = w_{\text{dup}}\,[x_i = x_j \neq \text{IDLE}]$ is the soft form of (1). Given its task, an agent moves to the cell of its region maximising (posterior mass in its sensing window)$/(1 + 0.1\, d_i)$; a rescuer moves to $\hat{v}$ and, once carrying, to the exit; IDLE agents hold position (this is what drones do once the belief has collapsed, saving movement).

4.4 Centralised baseline: exact assignment

A coordinator receives every scan every step (counted as one uplink message per agent per step, plus one downlink per changed assignment, but never limited), maintains one pooled belief and terrain map, and every 5 steps (or when a claim appears) solves $\min \sum_i u_i(x_i)$ subject to (1)–(2) *exactly*: rows are agents, columns the non-idle tasks plus one private IDLE column per agent, infeasible pairs get cost 10^6, and the rectangular assignment is solved by the Hungarian method [9] (SciPy `linear_sum_assignment`). Its optimality is checked against brute-force enumeration of all feasible assignments, and constraint (1) is never violated (both verified in `tests/test_model.py`).

4.5 Decentralised DCOP coordination: MGM and DSA-C

The review left the DCOP solver open. We implement two local-search solvers, MGM [7] (default) and DSA-C [8], rather than ADOPT [5]: ADOPT is complete but needs a DFS tree and a number of messages exponential in the worst case, which cannot be stopped at an arbitrary budget, whereas MGM is an anytime local search whose cost is linear in rounds, whose solution quality never decreases between rounds, and which degrades gracefully when messages are dropped—the regime we study. Every 5 steps each agent runs the rounds of Algorithm 1 using *its own* belief; tasks of neighbours are remembered in an *agent view* that expires after 15 steps. Between rounds, an agent hit by a new claim re-decides locally. With $B = 0$ no `VALUE`/`GAIN` message is ever sent and the algorithm reduces to independent greedy choices—the no-communication ablation. **DSA-C** uses the same `VALUE` phase but no `GAIN` messages: in each round every agent moves to its best response with probability $p = 0.7$ if this strictly lowers its local cost, or

Algorithm 1 One MGM allocation epoch at agent i (decentralised; all agents run it in parallel)

input: belief b_i, claim $\hat{v}$ (if any), agent view $\mathcal{V}_i$, quota left Q_i
compute unary costs $u_i(\cdot)$ on own domain; if current x_i infeasible: $x_i \leftarrow \arg\min_x [u_i(x) + \sum_{j \in \mathcal{V}_i} w(x, x_j)]$
for round $= 1..3$ **do**
  if x_i changed or announcement is stale **and** $Q_i >$ reserve: broadcast $\texttt{VALUE}(x_i)$ (cost 1)
  on receiving $\texttt{VALUE}(x_j)$: $\mathcal{V}_i[j] \leftarrow (x_j, t)$
  $x_i^* \leftarrow \arg\min_x \left[u_i(x) + \sum_{j \in \mathcal{V}_i,\,\text{fresh}} w(x, x_j)\right]$; $g_i \leftarrow \text{cost}(x_i) - \text{cost}(x_i^*)$
  if $g_i > 0$ **and** $Q_i >$ reserve: broadcast $\texttt{GAIN}(g_i)$ (cost 1)
  if $g_i > 0$ and $(g_i, -i) > (g_j, -j)$ for every gain g_j heard this round: $x_i \leftarrow x_i^*$
  if no agent changed: **break**

Table 1: Default parameters. Reliability is $(p^{\text{det}}, p^{\text{fa}})$ per scanned cell.

Grid / regions	20×20 / 4×4	Team	3 drones + 2 robots
Speed (drone / robot)	2 / 1 cells per step	Sensing radius (drone / robot)	2 / 1
High reliability	drone (0.95, 0.01), robot (0.70, 0.02)	Comm. range r_c	9 cells
Medium (default)	drone (0.80, 0.03), robot (0.55, 0.04)	Claim threshold τ	0.7
Low reliability	drone (0.60, 0.06), robot (0.45, 0.07)	Replan period / DCOP rounds	5 steps / 3
$w_{\text{info}}, w_{\text{res}}, w_{\text{dup}}$	400, 1000, 1000	Share period / reserve	5 steps / 2 msgs

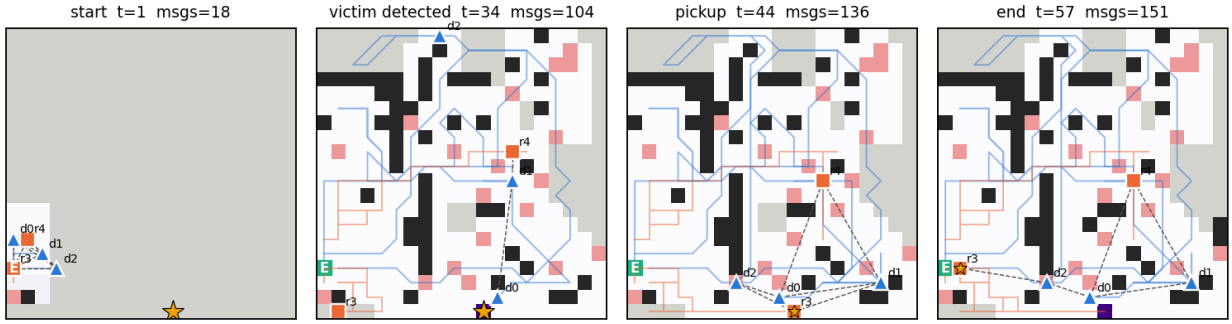


Figure 1: A decentralised episode (seed 3, $B = \infty$) from the simulator: obstacles (black), hazards (red), exit (E), drones d_j (blue triangles), robots r_j (orange squares), victim (star), communication links within range (dashed), agent trajectories, drone d_0's posterior (purple) and the cells d_0 has not discovered yet, directly or through messages (grey). The victim is claimed at $t = 34$, picked up by robot r_3 at $t = 44$ and carried out at $t = 57$.

keeps it equal while its current value conflicts with a neighbour's. It is cheaper per round than MGM but not monotone. Finally, a **no-coordination ablation** sends only `OBS` messages (tasks are learned from their piggyback) and lets every agent take its best response, isolating the value of coordination messages from the value of shared observations.

4.6 Experimental setup (`experiments/run_experiments.py`)

Default parameters are listed in Table 1. Each configuration is run on the same 300 seeds (matched building instances) for every method and budget; we report means with 95% confidence intervals. The objective weights are $\lambda_t = 1$, $\lambda_c = 0.1$, $\lambda_m = 0.05$, $\lambda_f = 300$ and the step limit (survival window) is 150. Experiments: **(E1)** budget $B \in \{0, 10, 20, 40, 80, 160, 320, \infty\}$ for MGM, DSA-C and the no-coordination ablation; **(E2)** E1 at three sensor-reliability levels; **(E3)** team size $(|\mathscr{D}|, |\mathscr{R}|) \in \{(2, 1), (3, 2), (5, 3)\}$ and **(E4)** building complexity (simple 15^2, medium 20^2, complex 28^2 with increasing obstacle/hazard density and walls), both at $B \in \{0, 20, 80, \infty\}$ (none/low/moderate/unrestricted); **(E5)** communication range $r_c \in \{4, 6, 9, 14, \infty\}$ at $B \in \{40, \infty\}$; **(E6)** known vs. unknown floor plan at $B \in \{0, 20, 80, \infty\}$. In total 30,600 episodes; the whole study runs in about 13 minutes on a 10-core laptop. Besides the four metrics of the review we log the handoff time (first step a robot holds a correct claim), the number of steps in which (1) is violated, robot trips to false claims, and *redundancy*: the fraction of scanned cells scanned by two or more agents.

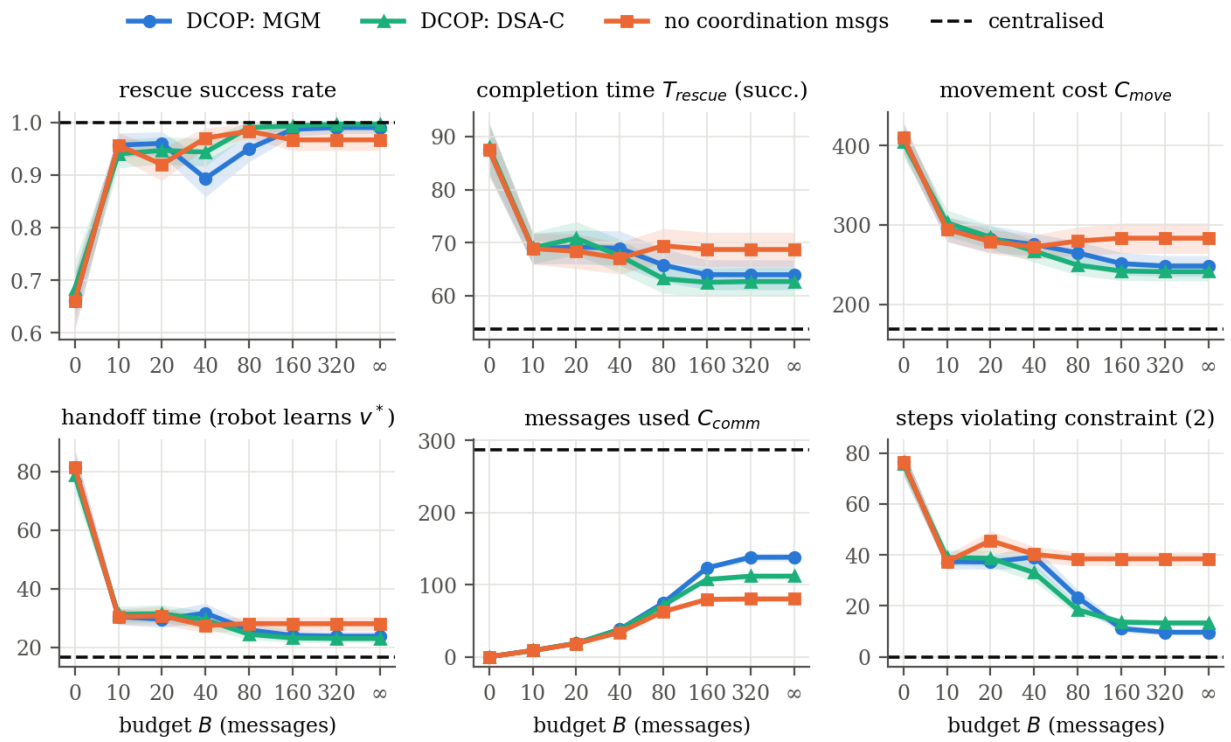


Figure 2: E1: metrics against the team budget B (unknown map, medium reliability, 300 seeds; bands are 95% CIs). Dashed line: centralised reference. Completion time is averaged over successful runs.

Table 2: E1 summary (mean $\pm$ 95% CI over 300 matched seeds). T on successful runs; "(2) viol." = steps in which two agents hold the same non-idle task..

Method	B	Success	T_{rescue}	C_{comm}	C_{move}	(2) viol.	Redund.
Centralised	–	1.00 ± 0.00	53.6 ± 2.1	287.2 ± 10.7	168 ± 6	0.0	0.49
DCOP: MGM	0	0.66 ± 0.05	87.5 ± 4.9	0.0 ± 0.0	410 ± 19	76.6	0.84
	10	0.96 ± 0.02	68.8 ± 2.9	8.6 ± 0.2	294 ± 15	37.3	0.75
	20	0.96 ± 0.02	69.2 ± 3.2	18.6 ± 0.2	282 ± 17	37.2	0.75
	40	0.89 ± 0.03	68.9 ± 3.3	38.0 ± 0.2	275 ± 14	39.2	0.70
	80	0.95 ± 0.02	65.8 ± 3.1	74.7 ± 0.5	265 ± 14	23.3	0.68
	∞	0.99 ± 0.01	63.9 ± 2.8	138.0 ± 5.5	248 ± 13	9.6	0.67
DCOP: DSA-C	10	0.94 ± 0.03	69.0 ± 2.9	8.7 ± 0.2	303 ± 16	39.0	0.76
	40	0.94 ± 0.03	67.5 ± 3.1	37.6 ± 0.3	267 ± 14	33.0	0.71
	∞	1.00 ± 0.01	62.6 ± 2.7	111.7 ± 4.3	241 ± 12	13.3	0.68
No coordination	10	0.96 ± 0.02	68.8 ± 2.9	8.6 ± 0.2	294 ± 15	37.3	0.75
	40	0.97 ± 0.02	67.0 ± 3.0	34.0 ± 0.4	272 ± 15	40.2	0.75
	∞	0.97 ± 0.02	68.7 ± 3.2	80.2 ± 3.8	283 ± 19	38.5	0.75

5 Results

5.1 E1: performance against the communication budget

Figure 2 and Table 2 answer the main question. Performance does **not** degrade gradually: it collapses only at the very bottom of the budget range. Without communication the team succeeds in 66% of runs, robots learn the victim location only after 81 steps on average (they must find the victim with their own poor sensors, although drones have found it by step 26) and two agents share a task in 77 of the steps. Allowing just $B = 10$ messages—two per agent, which the reserve rule spends on the claim and its relay—cuts the handoff time to 30 steps and raises success to 0.96, recovering 90% of the success gap to unrestricted communication. Beyond that, returns are small and, surprisingly, not monotone (see below); the budget stops binding at about 138 messages for MGM and

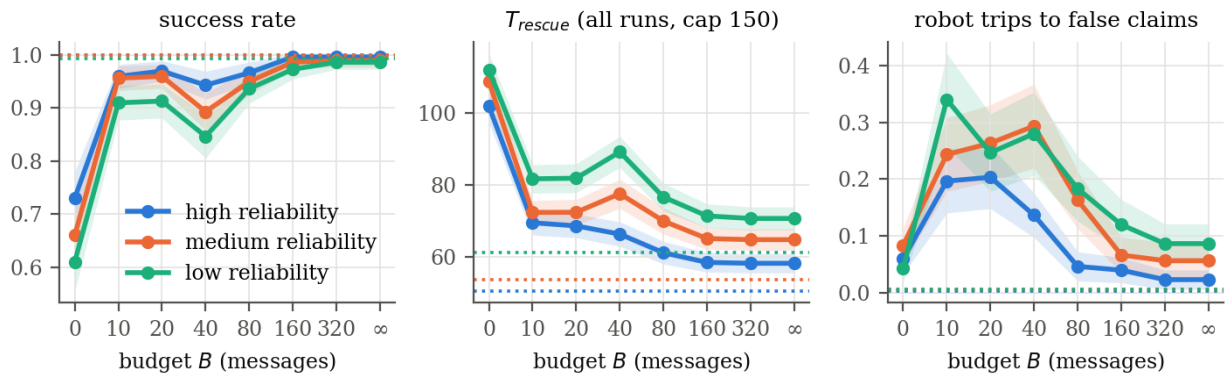


Figure 3: E2: budget sweep (MGM) at three sensor-reliability levels. Dotted lines: centralised reference of the same colour.

Table 3: E2: effect of sensor reliability (MGM). Success rate and T_{rescue} (all runs, failures counted at the 150-step limit); last two columns at $B = \infty$.

	Success				T_{rescue}				wrong	false
Reliab.	Central	$B{=}\infty$	$B{=}20$	$B{=}0$	Central	$B{=}\infty$	$B{=}20$	$B{=}0$	claims	trips
high	1.00	1.00	0.97	0.73	50.5	58.2	68.6	101.9	0.20	0.02
medium	1.00	0.99	0.96	0.66	53.6	64.8	72.4	108.8	0.56	0.06
low	0.99	0.99	0.91	0.61	61.2	70.7	81.9	112.0	0.66	0.09

112 for DSA-C. With ample budget, communication also removes redundant exploration (cells scanned by several agents fall from 84% to 67%) and cuts movement by 40%.

Gap to the centralised reference. With unrestricted budget the MGM team succeeds in 99% of runs but is 21% slower ($T = 64.8$ vs. 53.6 over all runs), moves 47% more and still has some duplicated assignments (10 steps), because evidence and terrain travel only through range-limited broadcasts and allocation is a local optimum. It uses 48% of the centralised message volume (138 vs. 287). E6 shows that about half of this time gap comes from the unknown map.

MGM vs. DSA-C vs. no coordination. With a generous budget, coordination messages clearly help: duplicated assignments fall from 38 steps (no coordination) to 10 (MGM) and 13 (DSA-C), and completion time from 68.7 to 63.9 and 62.6. DSA-C, which sends no gain messages, reaches the same quality with 19% fewer messages (112 vs. 138). Under a tight budget the picture reverses: at $B = 40$ the uncoordinated variant succeeds most often (0.97, vs. 0.94 for DSA-C and 0.89 for MGM). At $B = 20$ MGM spends 9.4 of its 18.6 messages on `VALUE`/`GAIN` and only 9.2 on observations, whereas the ablation spends all 18.2 on observations. The MGM dip at $B = 40$ comes with more wrong claims (0.83 vs. 0.59 at $B = 20$) and more robot trips to empty claimed cells: agents spend their few non-reserved messages on early, partial evidence and coordination, so some claims form on evidence that is never refuted. When messages are scarce, sharing evidence is worth more than negotiating who searches where.

5.2 E2: is communication more valuable when sensors are noisy?

Yes (Figure 3, Table 3). The success gap between no communication and unrestricted communication grows from 0.27 (high reliability) to 0.33 (medium) and 0.38 (low), and the robots' wait for the victim location without communication grows from 71 to 84 steps. With noisy sensors no single agent can confirm the victim quickly, so pooling evidence across agents matters more. The review's secondary hypothesis—that sharing also spreads false alarms—is confirmed in a specific form (right panel of Figure 3): robot trips to empty claimed cells peak at *small and intermediate* budgets ($B = 10$–40), where a claim is transmitted but most of the evidence that would later refute it is not, and they vanish as B grows. Under low reliability this again produces a dip at $B = 40$ (success 0.85, vs. 0.91 at $B = 10$): a little noisy information beyond the victim report can be worse than none.

5.3 E3–E5: is the trend specific to one team or one map?

The shape of the curve—large loss at $B = 0$, most of it recovered by a small budget—holds for every team size and map (Figure 4). **Team size:** the smallest team suffers most without communication (success 0.52 for 2D+1R vs. 0.80 for 5D+3R) because a single robot must find the victim alone. The centralised message volume grows linearly with team size (205→439) while the decentralised volume at $B = \infty$ stays below half of it (88→215). The largest team also shows the intermediate-budget dip (0.98 at $B = 20$, 0.90 at $B = 80$), with the most wrong claims (1.8 per run): more agents sharing partial noisy evidence produce more false claims. **Complexity:** the gap widens with the building. On the simple map even $B = 0$ succeeds in 95% of runs; on the complex 28×28 map success falls to 0.37 without communication, and the decentralised team stays well below the centralised one even with

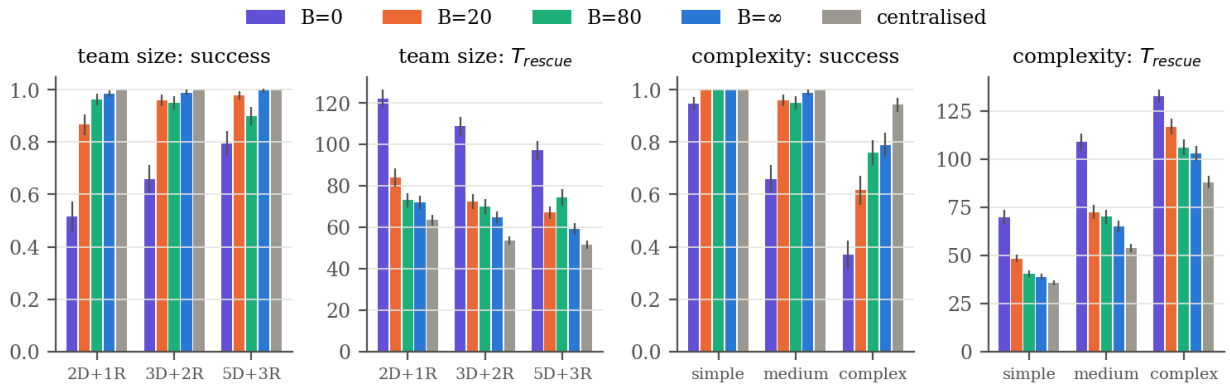


Figure 4: E3/E4: success and completion time across team sizes and building complexity at four budgets (MGM) and for the centralised reference (95% CIs).

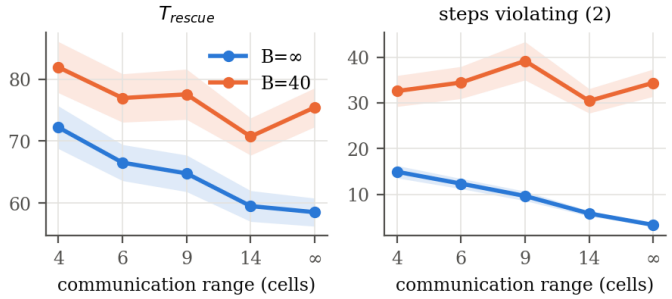


Figure 5: E5: completion time and constraint-(2) violations against communication range.

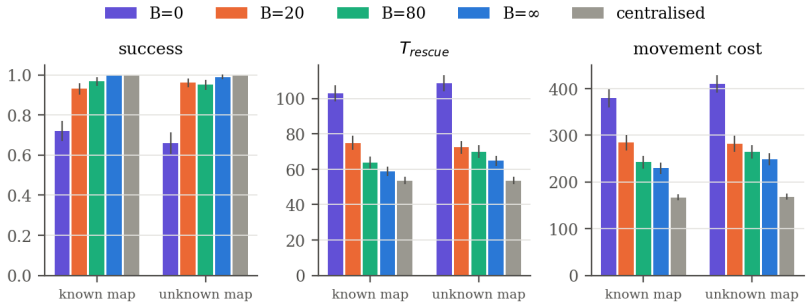


Figure 6: E6: known floor plan vs. unknown (discovered) floor plan at four budgets (MGM) and centralised.

unrestricted budget (0.79 vs. 0.94), because the fixed 9-cell range covers a smaller fraction of the building and there is more map to discover. **Communication range (Figure 5):** with $B = \infty$, widening the range steadily closes the gap ($T = 72.2$ at $r_c = 4$ to 58.5 with global broadcast vs. 53.6 centralised) and removes most duplicated assignments (15→3 steps); with $B = 40$ the budget, not connectivity, dominates (duplicates stay at 30–39 steps for every range).

5.4 E6: what does not knowing the floor plan cost?

The centralised coordinator is unaffected by having to discover the map ($T = 53.6$ either way): it pools every agent's terrain observations instantly. The decentralised team is not: with unrestricted budget its completion time rises from 58.8 to 64.8 (gap to centralised 10%→21%) and its movement by 8%; without communication success falls from 0.72 to 0.66 (Figure 6). Discovered terrain is therefore a second kind of information—after the victim report—that the budget must carry. Conversely, at low budgets the unknown map slightly *reduces* false claims (0.59 vs. 1.49 at $B = 20$), because the wider prior requires more evidence before a claim is made.

5.5 Discussion and limitations

The value of communication is concentrated in the few messages that carry the drone-to-robot handoff; without them complementary agent types are of little use. Further budget buys less redundant exploration, shared terrain and better coordination. More budget is not always better: at intermediate budgets partial, noisy evidence and coordination messages can do harm, which suggests that a budget-aware messaging rule—deciding between evidence, terrain and coordination messages—is the natural next step. Limitations: a single victim; static hazards; independent sensor noise; lossless messages within range; optimistic planning on unknown cells; MGM and DSA-C reach local optima only; results depend on our cost weights and are simulation results, not claims about real emergency-response systems.

6 Contribution

All tasks assigned in the project review were completed; each member also contributed to integration, testing, debugging and writing.

- **Boinapally Dhanvini** — literature review (multi-agent SAR, heterogeneous teams, CSP/DCOP) and the formal model of Section 3; choice of MGM/DSA-C over ADOPT and derivation of the task set and cost functions from (4). *Status: fulfilled.*
- **Bollineni Snehitha Rao** — environment and agent modelling: building generation, obstacles, hazards, movement, the observation model (3) and reliability/complexity settings. She also built an independent prototype whose two key ideas—a floor plan discovered by sensing and DSA-C allocation—were integrated into the final code (`sar/mapknow.py`, DSA-C in `sar/sim.py`). *Status: fulfilled.*
- **Jils Shah** — centralised Hungarian baseline and decentralised MGM/DSA-C coordination (`sar/allocation.py`, `sar/sim.py`); communication budget, messaging rule and identical message accounting for all methods; integration of the final simulator; unit tests checking constraints (1)–(2), the budget and the optimality of the baseline. *Status: fulfilled.*
- **Thaker Shivangi Pareshkumar** — experimental design and analysis (`experiments/`): the six experiments with matched seeds, metrics, plots and tables, and the interpretation in Section 5. *Status: fulfilled.*

7 Conclusion

We built a simulator of heterogeneous search and rescue in a building whose layout and victim location are unknown, under a communication budget, and compared an exact centralised task allocation with decentralised MGM and DSA-C over 30,600 matched episodes. Performance falls sharply only when communication is (almost) removed: two messages per agent, used for the drone-to-robot handoff, recover 90% of the lost success, and an unrestricted decentralised team nearly matches the centralised success rate with 48% of its messages, at a 21% time cost of which half is due to discovering the map. Communication matters more when sensors are noisy, for smaller teams and in larger buildings; at small budgets observation sharing should take priority over coordination messages, and DSA-C is the more message-efficient coordinator. Code, data, figures and instructions to reproduce every number are in the repository.

References

[1] J. Peña Queralta, J. Taipalmaa, B. Can Pullinen, V. K. Sarker, T. Nguyen Gia, H. Tenhunen, M. Gabbouj, J. Raitoharju, and T. Westerlund. Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision. *IEEE Access*, 8:191617–191643, 2020.

[2] F. Fioretto, E. Pontelli, and W. Yeoh. Distributed constraint optimization problems and applications: A survey. *Journal of Artificial Intelligence Research*, 61:623–698, 2018.

[3] T. Gunn and J. Anderson. Dynamic heterogeneous team formation for robotic urban search and rescue. *Journal of Computer and System Sciences*, 81(3):553–567, 2015.

[4] G. A. Korsah, A. Stentz, and M. B. Dias. A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research*, 32(12):1495–1512, 2013.

[5] P. J. Modi, W.-M. Shen, M. Tambe, and M. Yokoo. ADOPT: Asynchronous distributed constraint optimization with quality guarantees. *Artificial Intelligence*, 161(1–2):149–180, 2005.

[6] W. Burgard, M. Moors, C. Stachniss, and F. E. Schneider. Coordinated multi-robot exploration. *IEEE Transactions on Robotics*, 21(3):376–386, 2005.

[7] R. T. Maheswaran, J. P. Pearce, and M. Tambe. Distributed algorithms for DCOP: A graphical-game-based approach. In *Proc. ISCA PDCS*, pages 432–439, 2004.

[8] W. Zhang, G. Wang, Z. Xing, and L. Wittenburg. Distributed stochastic search and distributed breakout: properties, comparison and applications to constraint optimization problems in sensor networks. *Artificial Intelligence*, 161(1–2):55–87, 2005.

[9] H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.